

OBJECT-RELATIONAL MAPPING

ORM: aproximando *objetos* e banco de dados

Como o código orientado a objetos conversa com as tabelas de um banco relacional

objeto

Aluno
id
nome
turma

tabela

aluno	
id	PK
nome	
turma_id	FK

ORM

O caminho que vamos *percorrer*

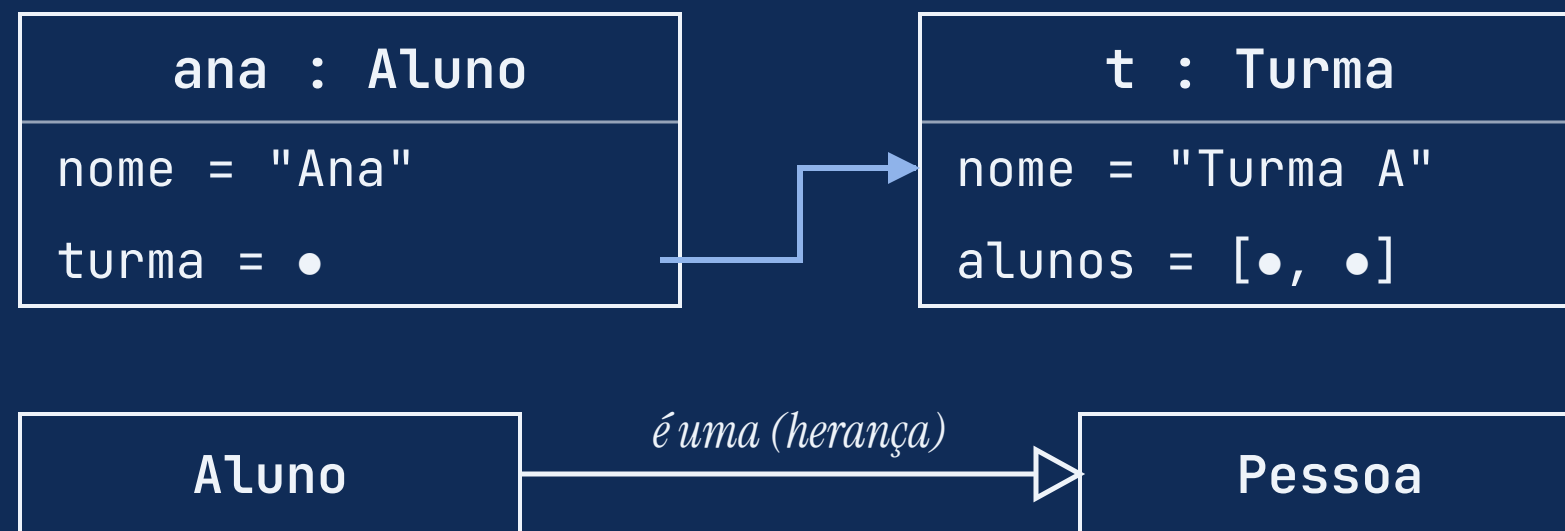
Pergunta-guia: como um objeto vira uma linha, e volta?

01 SLIDE 03 O problema Dois mundos diferentes	02 SLIDE 08 O que é ORM A ponte entre eles	03 SLIDE 13 Entidades Classes viram tabelas	04 SLIDE 18 Atributos Campos viram colunas
05 SLIDE 22 Relacionamentos Referências viram chaves	06 SLIDE 31 Além do básico Herança e padrões	07 SLIDE 34 Demonstração Matrícula escolar, ao vivo	08 SLIDE 37 Balanço Prós, contras, quando usar

PARTE 01 O problema

Dois mundos que pensam *diferente*

Mundo dos objetos Java · C# · Python

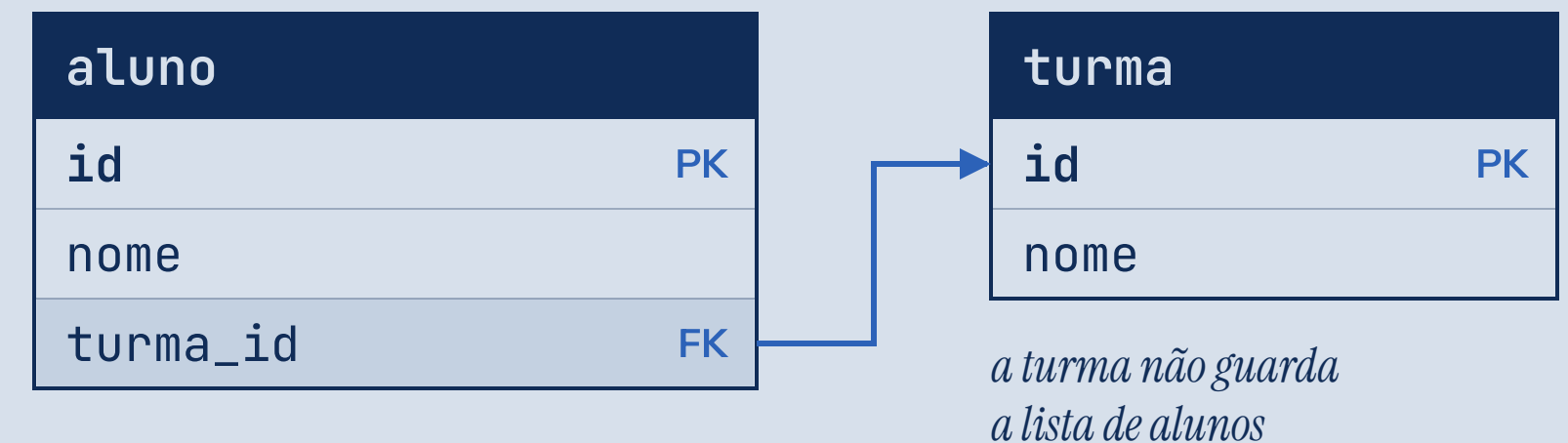


- Classes com dados e comportamento
- Objetos apontam uns para os outros
- Herança e listas de objetos

IMAGINE

Objetos são os contatos do celular, com links entre si; tabelas são fichas numeradas num arquivo de aço.

Mundo relacional SQL



- Tabelas só com valores simples
- Linhas ligadas por chaves estrangeiras
- Sem herança; lista vira outra tabela

O ORM é o *intérprete simultâneo* entre a aplicação e o banco de dados

IDA · A PERGUNTA

01 · APLICAÇÃO

Fala “*objetês*”

```
aluno.getTurma()  
  .getNome()
```

02 · ORM

Traduz

Sabe que o atributo **turma** é a coluna `turma_id` e monta o JOIN com a tabela **turma**.

03 · BANCO

Fala *SQL*

```
SELECT t.nome  
FROM turma t  
JOIN aluno a  
  ON a.turma_id = t.id  
WHERE a.id = 1
```

VOLTA · A RESPOSTA

recebe o valor
"Turma A"

cria o objeto
Turma { nome = "Turma A" }

devolve a linha
nome = 'Turma A'

Ninguém precisa aprender a língua do outro. Mas quem entende um pouco das duas percebe quando a tradução sai errada.

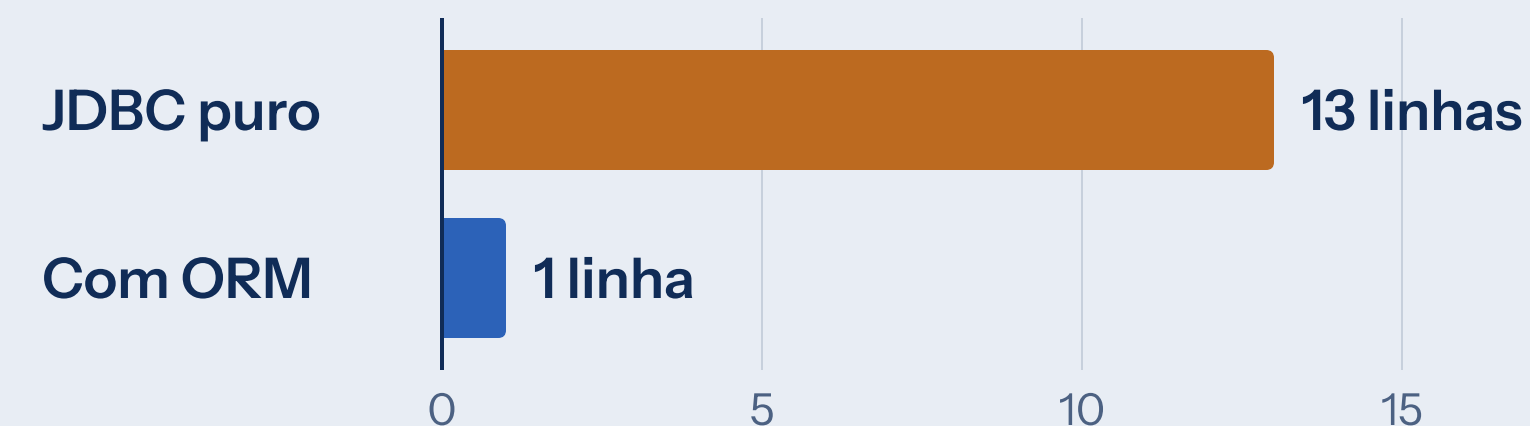
Sem ORM, a tradução é feita *à mão*

BUSCARALUNO.JAVA

JDBC PURO

```
// cada campo é copiado à mão
String sql = "SELECT id, nome, email"
    + " FROM aluno WHERE id = ?";
PreparedStatement ps =
    conn.prepareStatement(sql);
ps.setLong(1, 42);
ResultSet rs = ps.executeQuery();
Aluno a = null;
if (rs.next()) {
    a = new Aluno();
    a.setId(rs.getLong("id"));
    a.setNome(rs.getString("nome"));
    a.setEmail(rs.getString("email"));
}
```

Linhas de código para buscar *1 aluno*



Linhas de código deste exemplo, sem o comentário. Com ORM:
Aluno a = em.find(Aluno.class, 42L);

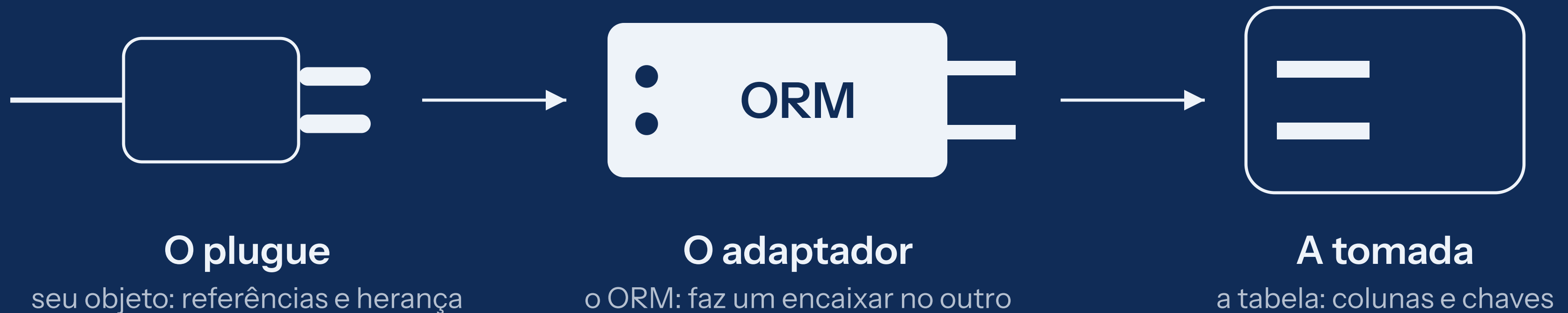
- O mesmo código se repete em cada classe
- Coluna com nome errado só falha ao rodar
- Mudou a tabela? Caçar todos os SQLs

O descompasso de *impedância*

DIFERENÇA	NO MUNDO DOS OBJETOS	NO BANCO RELACIONAL
Granularidade	Muitas classes pequenas: Cliente tem um Endereco	Poucas tabelas largas: endereço vira colunas de cliente
Herança	Aluno e Professor herdam de Pessoa	Não existe herança entre tabelas
Identidade	Mesmo objeto (==) ou mesmo conteúdo (equals)	Mesma chave primária
Associações	Referência com direção: aluno.turma	Chave estrangeira sem direção: turma_id
Navegação	De objeto em objeto: a.getTurma()	Conjuntos de uma vez, com JOIN

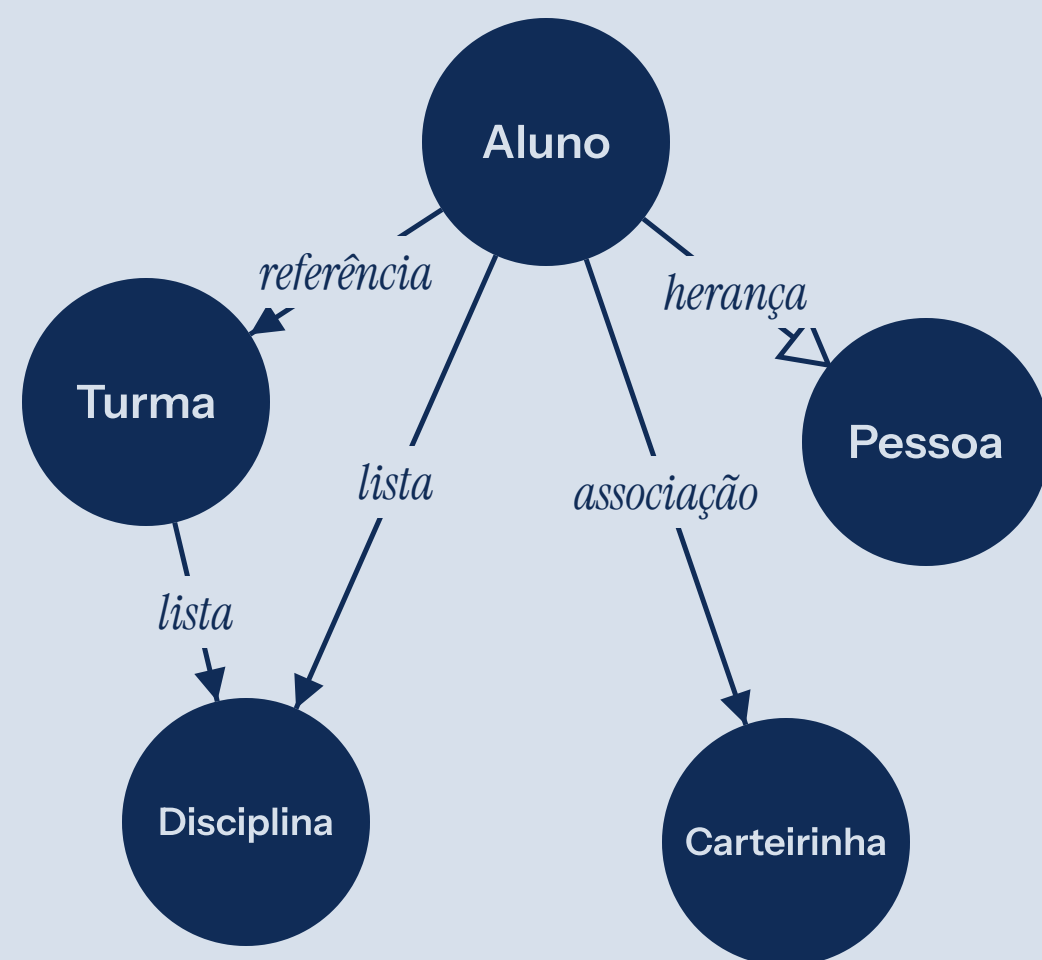
Por que o nome “*impedância*”?

Vem da engenharia elétrica: com impedâncias diferentes, a energia não passa bem de um circuito para o outro. É como levar um carregador de pinos redondos para um país de pinos chatos: **o encaixe não acontece**.



O adaptador resolve o encaixe, mas não muda a voltagem: o ORM esconde as diferenças, não as elimina.

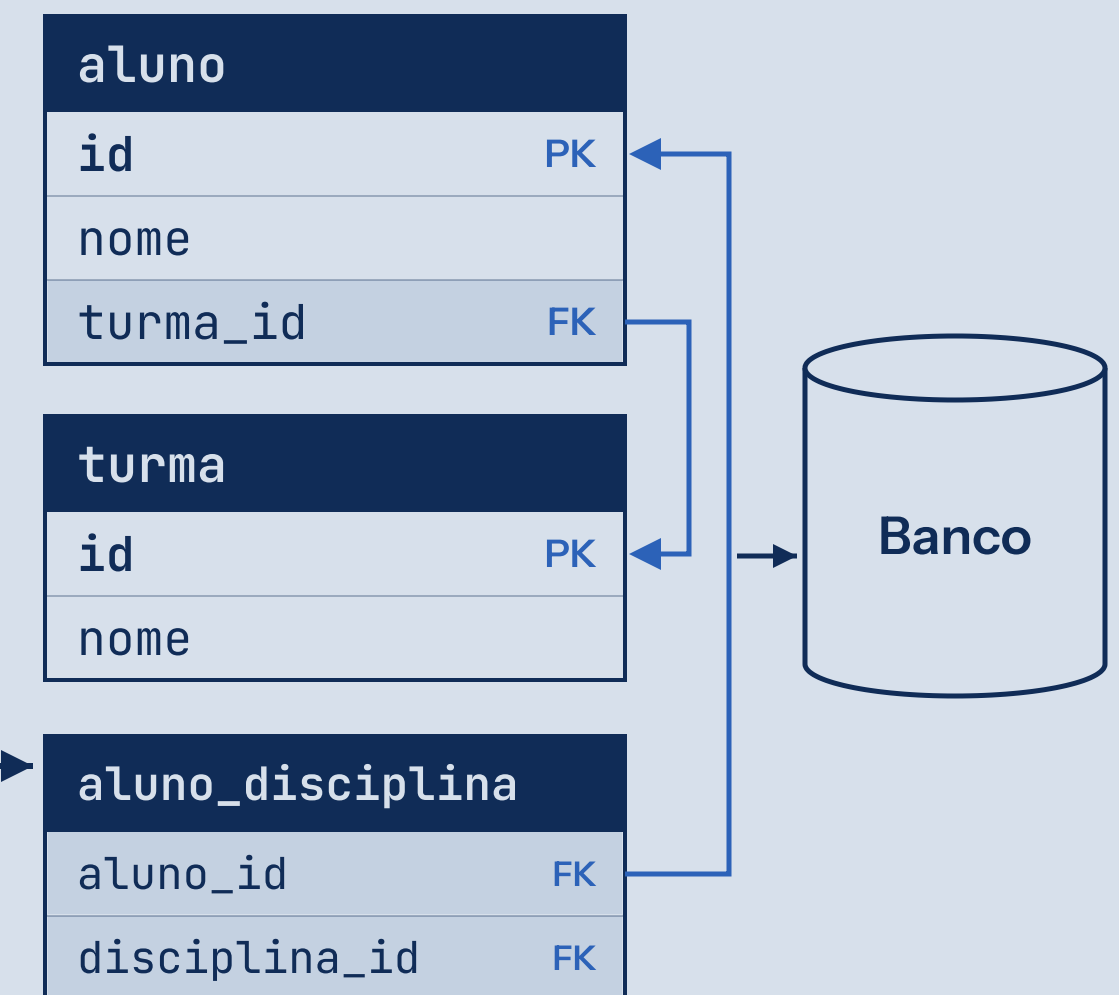
PARTE 02 O que é ORM

ORM: a *ponte* objeto-relacional

Mundo dos objetos

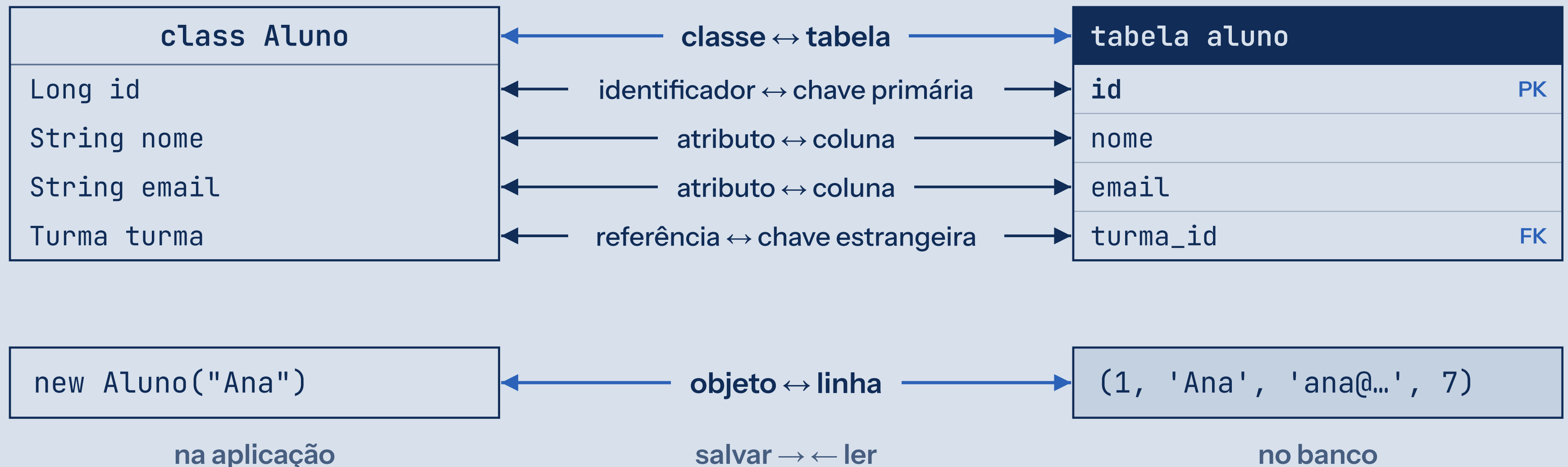


objeto → linha · linha → objeto



Mundo relacional

ORM é a técnica que *mapeia* classes para tabelas e converte sozinho objetos em linhas (ao salvar) e linhas em objetos (ao ler)



Um objeto vira *uma linha*

1 · VOCÊ ESCRIVE JAVA

```
Aluno ana = new Aluno();
ana.setNome("Ana Souza");
ana.setEmail("ana@uni.br");
ana.setCurso("SI");
em.persist(ana);
```

2 · O ORM GERA O SQL

```
INSERT INTO aluno
  (nome, email, curso)
VALUES
  ('Ana Souza',
   'ana@uni.br', 'SI');
```

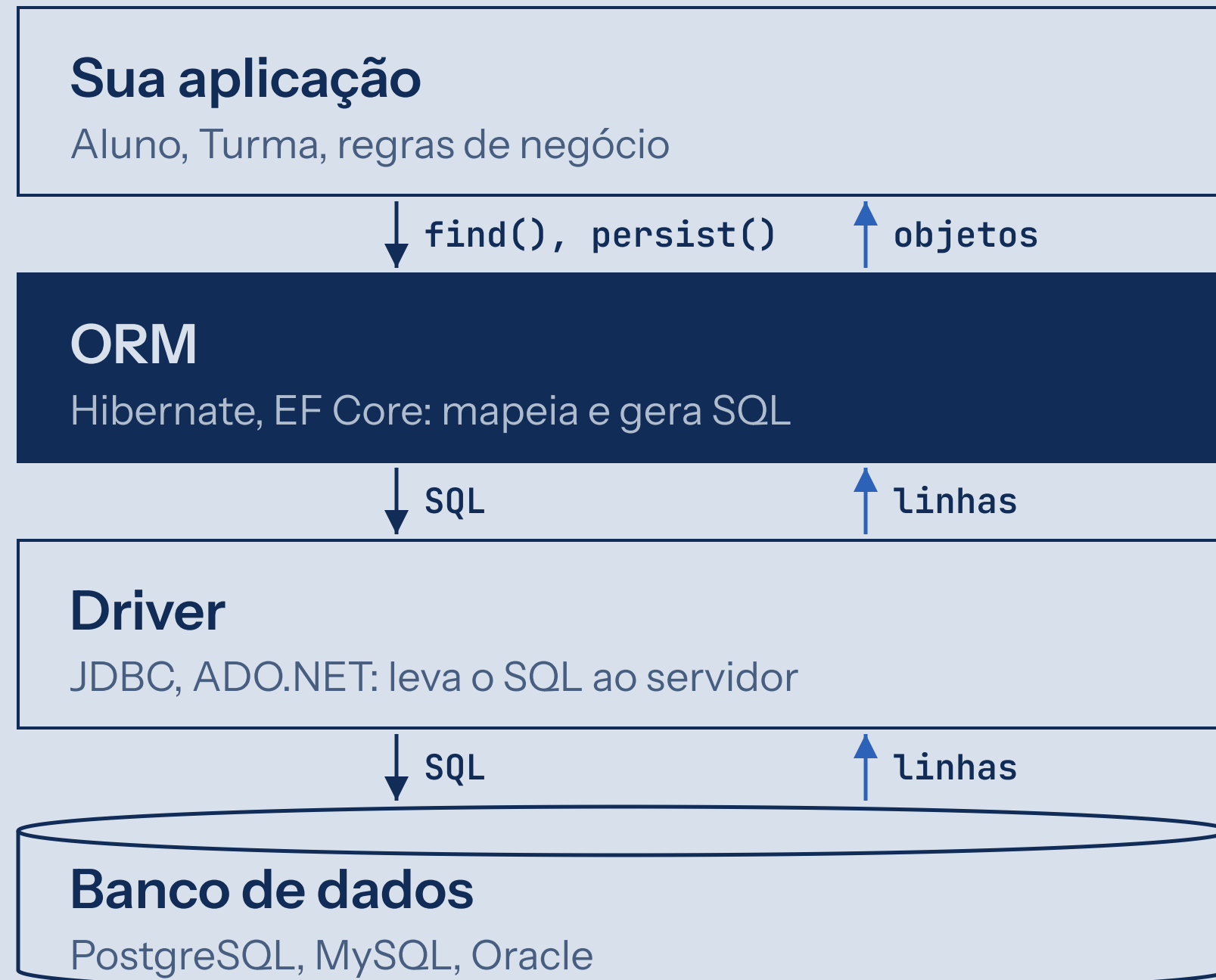
3 · A TABELA GANHA UMA LINHA

id	nome	curso
1	Bruno Lima	ADS
2	Carla Dias	SI
3	Ana Souza	SI

O **id 3** foi gerado pelo banco e copiado para o objeto: depois do `persist`, `ana.getId()` devolve **3**.

VOLTA `em.find(Aluno.class, 3L) → SELECT ... WHERE id = 3 → objeto Aluno preenchido`

Onde o ORM *mora* no sistema



O que o ORM *faz por você*

- 01 Lê o mapeamento (anotações ou XML)
- 02 Gera INSERT, SELECT, UPDATE, DELETE
- 03 Converte tipos: LocalDate ↔ DATE
- 04 Guarda objetos já lidos (cache)
- 05 Percebe mudanças (*dirty checking*)
- 06 Adapta o SQL a cada banco
- 07 Controla transações: tudo ou nada

ORMs que você vai *encontrar*

Hibernate Implementa a JPA; também existe o EclipseLink JAVA DATA MAPPER	EF Core Oficial da Microsoft; consultas com LINQ C# / .NET DATA MAPPER	SQLAlchemy Flexível; o Django tem ORM próprio PYTHON DATA MAPPER
Prisma Gera um cliente tipado a partir de um schema TYPESCRIPT SCHEMA	Active Record O ORM do Rails, que deu nome ao padrão RUBY ACTIVE RECORD	Eloquent O ORM do Laravel; o Doctrine é a alternativa PHP ACTIVE RECORD

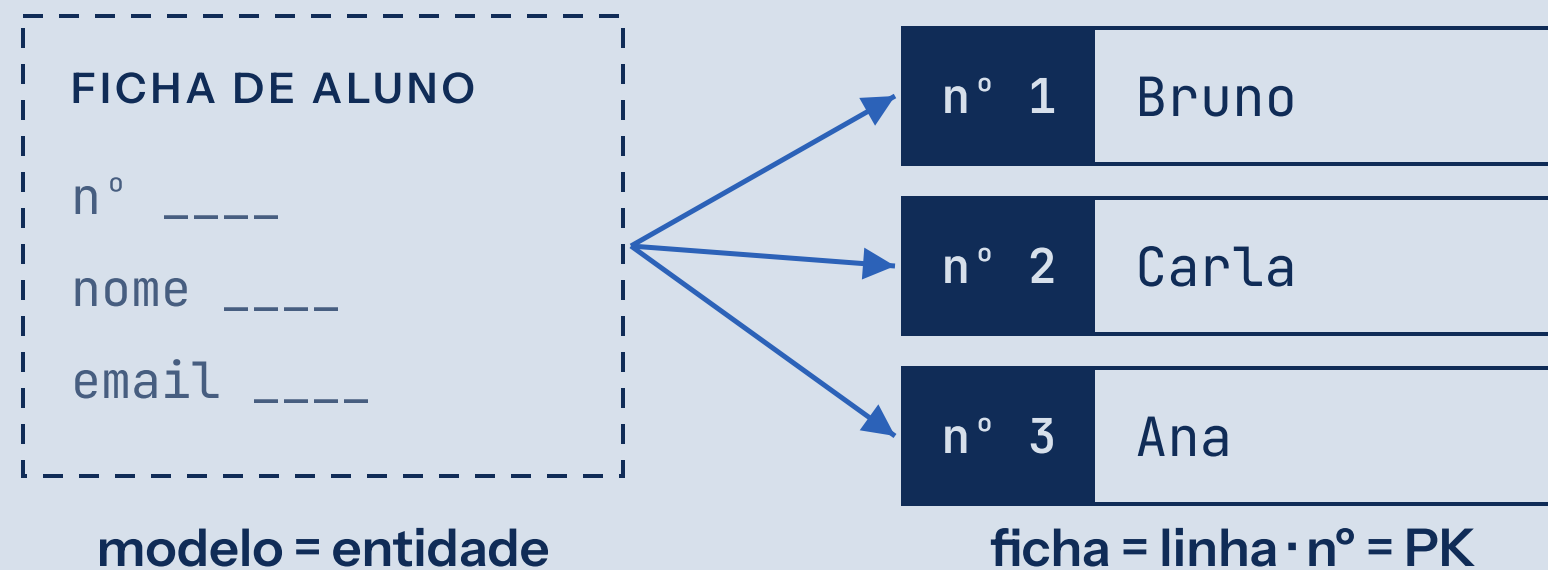
JPA é a regra; o Hibernate é quem a cumpre. Daqui em diante: Java + JPA.

PARTE 03 Entidades

O que é uma *entidade*?

Uma classe cujos objetos precisam ser *guardados* e têm *identidade própria*: cada um é único e pode ser encontrado de novo.

ALEGORIA · A SECRETARIA



Regras de uma entidade JPA

- Anotada com @Entity
- Tem um identificador com @Id
- Tem construtor sem argumentos
- Não é final (o ORM cria proxies)

É *entidade*?

Sim: Aluno, Turma, Pedido, Produto

Não: Endereço (só um valor), um total calculado

A entidade no código e a *tabela gerada*

ALUNO.JAVA

JPA

```
@Entity
@Table(name = "aluno")
public class Aluno {
    @Id
    @GeneratedValue(strategy =
        GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false,
        length = 100)
    private String nome;
    @Column(unique = true)
    private String email;
    protected Aluno() { }
}
```

SQL GERADO

MYSQL

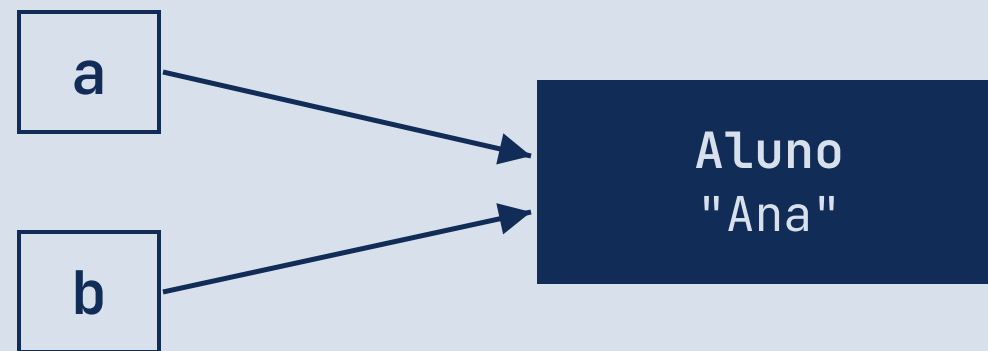
```
CREATE TABLE aluno (
    id      BIGINT AUTO_INCREMENT,
    nome    VARCHAR(100) NOT NULL,
    email   VARCHAR(255) UNIQUE,
    PRIMARY KEY (id)
);
```

@Entity	vai para o banco
@Table	nome da tabela
@Id	chave primária
@GeneratedValue	o banco gera o nº
@Column	detalhes da coluna

Identidade: como saber *quem é quem*

a == b

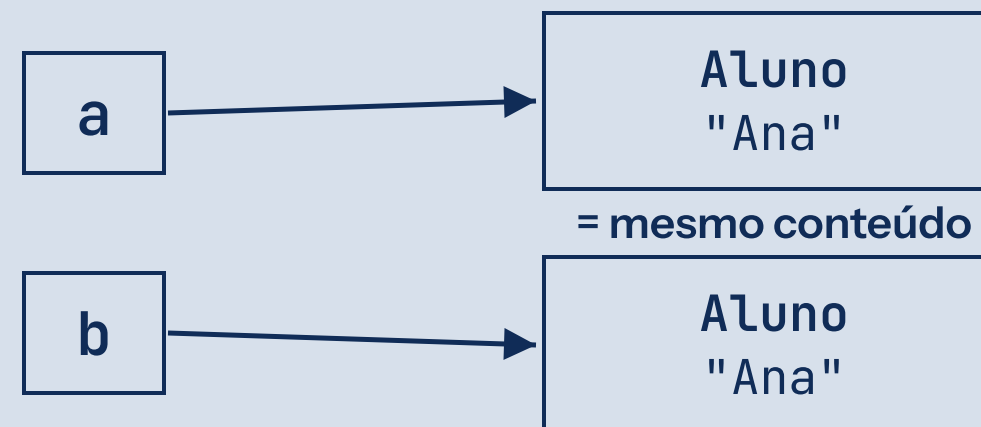
IDENTIDADE JAVA



O **mesmo objeto** na memória: duas variáveis, uma coisa só.

a.equals(b)

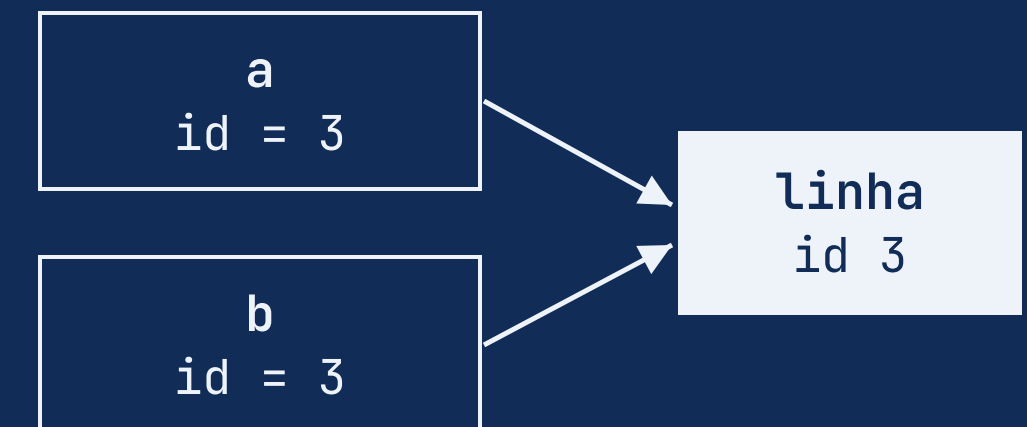
IGUALDADE



Mesmo conteúdo, como gêmeos: parecem iguais, mas são dois.

@Id

IDENTIDADE NO BANCO

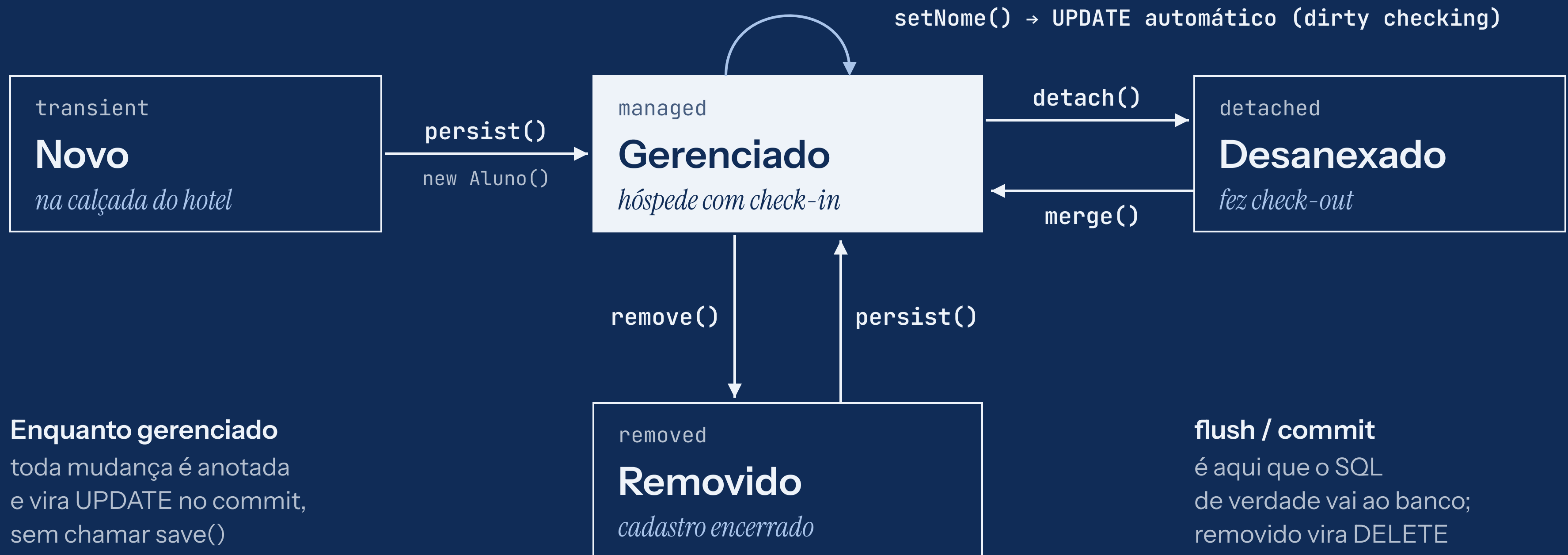


Mesma chave primária = mesmo registro. É esta que o ORM usa.

Quem escolhe o *número* da chave?

ESTRATÉGIA	COMO FUNCIONA	ALEGORIA
IDENTITY	Coluna auto-incremento do banco	<i>Senha da fila: o banco entrega o próximo número</i>
SEQUENCE	Objeto SEQUENCE do banco	<i>Talão de senhas, pedido antes de salvar</i>
TABLE	Tabela auxiliar guarda o próximo valor	<i>Caderno do porteiro: funciona, mas é lento</i>
UUID	Valor aleatório de 128 bits	<i>Código de rastreio: único sem combinar com ninguém</i>

O ciclo de vida de uma entidade



PARTE 04 Mapeamento de atributos

Cada tipo Java tem um *par* no SQL

JAVA	EXEMPLO	SQL
String	"Ana"	VARCHAR(255)
int / Integer	21	INTEGER
long / Long	1024	BIGINT
BigDecimal	1999.90	DECIMAL(10,2)
boolean	true	BOOLEAN / BIT
LocalDate	2026-09-16	DATE
LocalDateTime	... 20:14	TIMESTAMP
enum Status	0 ou "ATIVO"	INTEGER / VARCHAR
byte[]	foto	BLOB / BYTEA

CASA DE CÂMBIO

O valor é o mesmo, muda a “moeda”, sempre pela mesma tabela de conversão.

Dinheiro

Nunca double: BigDecimal ↔ DECIMAL
não perde centavos.

int × Integer

int nunca é null. Coluna aceita NULL?
Use Integer.

Ajustando cada *atributo*

<pre>@Column(name="nm_aluno", length=150, nullable=false)</pre> @ColumnCOLUNA Nome, tamanho, nulo, único	<pre>@Transient private int idade;</pre> @TransientIGNORAR Não vai para o banco	<pre>@Enumerated(EnumType.STRING) private Status status;</pre> @EnumeratedENUM Grava “ATIVO”, não a posição
<pre>@Lob private byte[] foto;</pre> @LobGRANDE Textos e arquivos grandes	<pre>// sem anotação! private LocalDate nascimento;</pre> java.timeDATAS Mapeado sozinho (JPA 2.2+)	<pre>@Convert(converter = CpfConverter.class)</pre> @ConvertPRÓPRIO Seu conversor (ex.: CPF)

Uma classe a mais, *nenhuma tabela nova*

CLIENTE.JAVA

JPA

```
@Embeddable
public class Endereco {
    private String rua;
    private String cidade;
    private String cep;
}

@Entity
public class Cliente {
    @Id @GeneratedValue
    private Long id;
    private String nome;
    @Embedded
    private Endereco endereco;
}
```

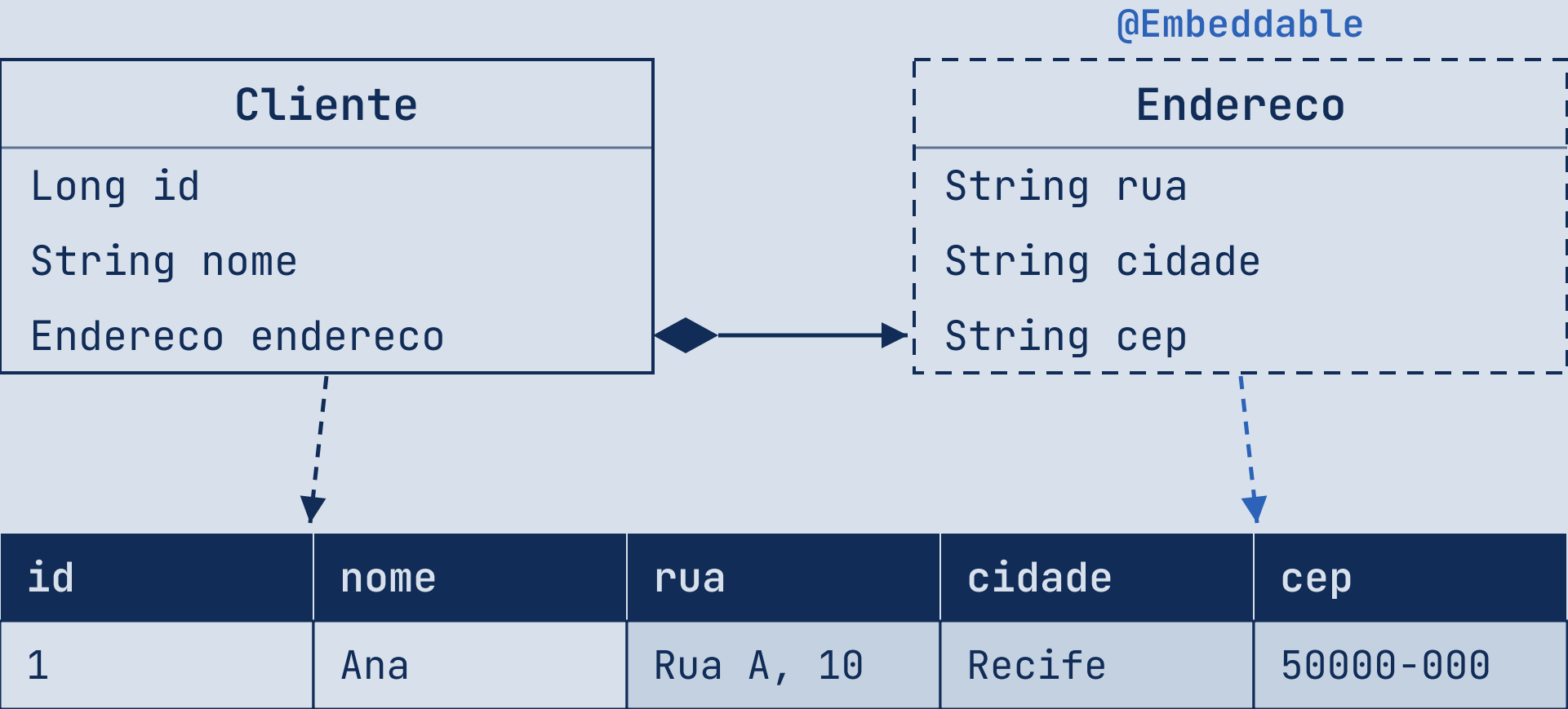


tabela cliente (uma só!)

ALEGORIA · A GAVETA

O endereço é uma gaveta: organizada, mas só existe dentro do armário (cliente). Sem @Id, sem tabela.

O ORM *adivinha o óbvio*; você só anota o que foge do padrão

NA CLASSE	O ORM ASSUME, SEM ANOTAÇÃO	ANOTE QUANDO...
<code>class Aluno</code>	tabela aluno	a tabela é TB_ALUNO
<code>nomeCompleto</code>	coluna nome_completo *	a coluna tem outro nome
<code>String email</code>	VARCHAR(255), aceita NULL	precisa de NOT NULL ou outro tamanho
<code>Long id</code>	nada: não adivinha a chave	sempre: @Id é obrigatório

ALEGORIA *A costureira já tem o molde M: você só pede ajuste onde o seu corpo é diferente.*

* Com a estratégia de nomes do Spring Boot. No Hibernate puro, a coluna fica nomeCompleto.

PARTE 05 Relacionamentos

Os *quatro* tipos de relacionamento

⌋ exatamente um ⌋ muitos (pé de galinha)

1:1

@OneToOne

Um aluno, uma carteirinha



banco → FK com UNIQUE

1:N

@OneToMany

Uma turma, muitos alunos



banco → FK fica em aluno

N:1

@ManyToOne

O mesmo, visto do aluno



banco → coluna aluno.turma_id

N:N

@ManyToMany

Alunos e disciplinas



banco → tabela de junção

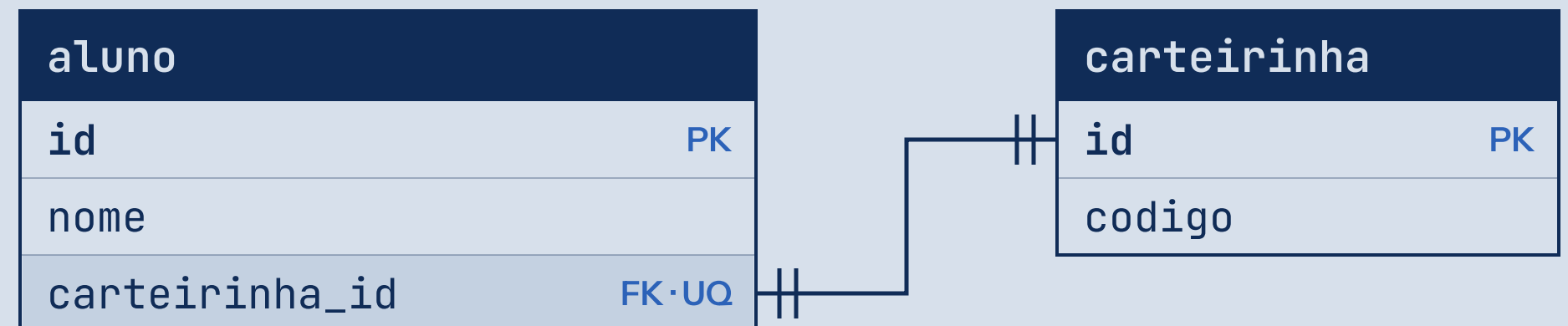
1:1 · um aluno, *uma* carteirinha

ALUNO.JAVA · CARTEIRINHA.JAVA

JPA

```
@Entity
public class Aluno {
    @Id @GeneratedValue
    private Long id;
    private String nome;
    @OneToOne
    @JoinColumn(name = "carteirinha_id",
                unique = true)
    private Carteirainha carteirinha;
}

@Entity
public class Carteirainha {
    @Id @GeneratedValue
    private Long id;
    private String codigo;
}
```



aluno

1	Ana	→	501
2	Bruno	→	502

carteirinha

501	UNI-0001
502	UNI-0002

- @JoinColumn: qual coluna guarda a FK
- unique = true: duas pessoas nunca dividem a mesma carteirinha
- Bom para dados opcionais ou pesados (foto, prontuário)

1:N · a turma e *seus alunos*

TURMA.JAVA

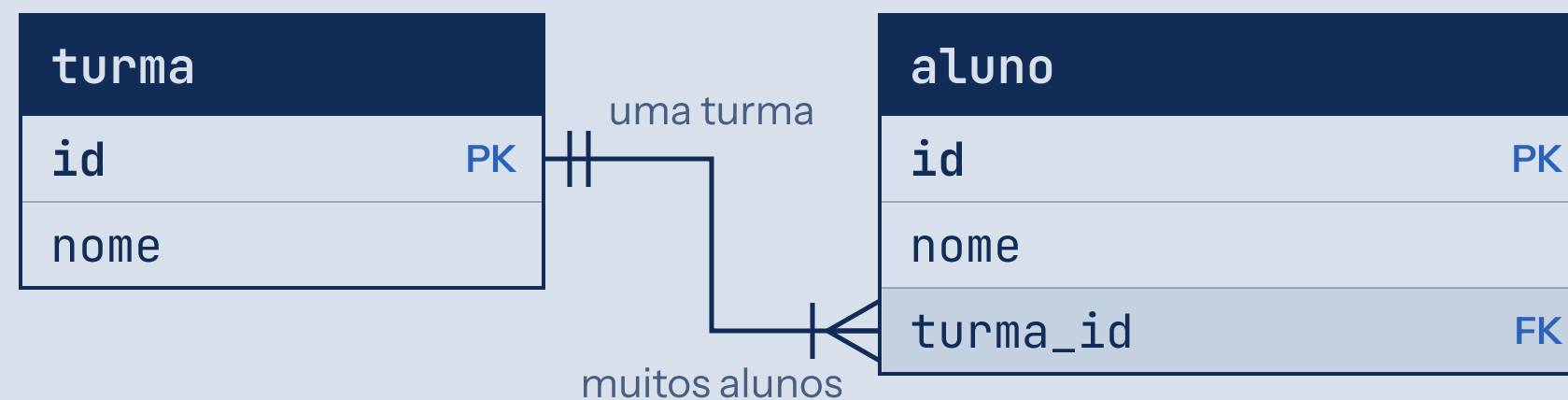
LADO “UM”

```
@Entity
public class Turma {
    @Id @GeneratedValue
    private Long id;
    @OneToMany(mappedBy = "turma")
    private List<Aluno> alunos;
}
```

ALUNO.JAVA

LADO “MUITOS” · DONO

```
@Entity
public class Aluno {
    @Id @GeneratedValue
    private Long id;
    @ManyToOne
    @JoinColumn(name = "turma_id")
    private Turma turma;
}
```



Quem é o *dono*?

Quem guarda a FK: **Aluno**. O `mappedBy` em `Turma` diz “o mapeamento está em `Aluno.turma`”, sem criar outra coluna.

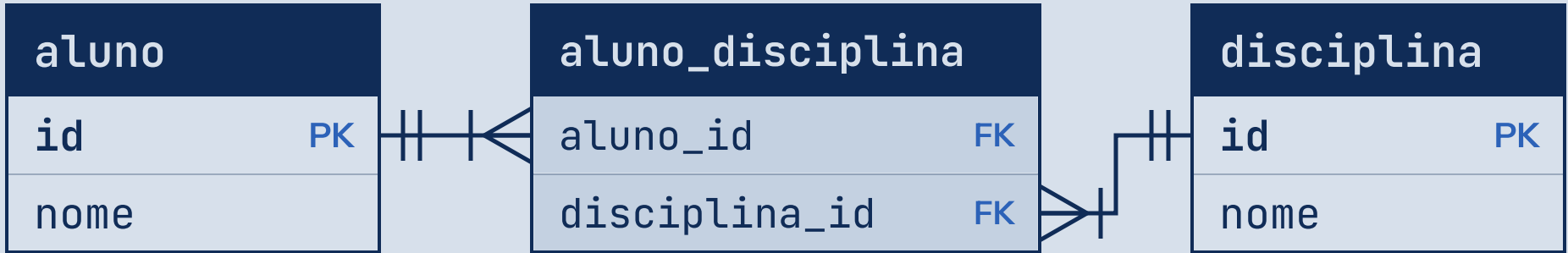
N:N · alunos e *disciplinas*

ALUNO.JAVA

DONO

```
@ManyToMany
@JoinTable(
    name = "aluno_disciplina",
    joinColumns =
        @JoinColumn(name = "aluno_id"),
    inverseJoinColumns =
        @JoinColumn(name = "disciplina_id"))
private Set<Disciplina> disciplinas;

// em Disciplina.java:
@ManyToMany(mappedBy = "disciplinas")
```



aluno_disciplina

aluno_id	disciplina_id
1 (Ana)	10 (BD)
1 (Ana)	20 (P00)
2 (Bruno)	10 (BD)

Precisa guardar *nota ou data*?

A junção vira a entidade **Matricula**, com dois @ManyToOne.

ALEGORIA

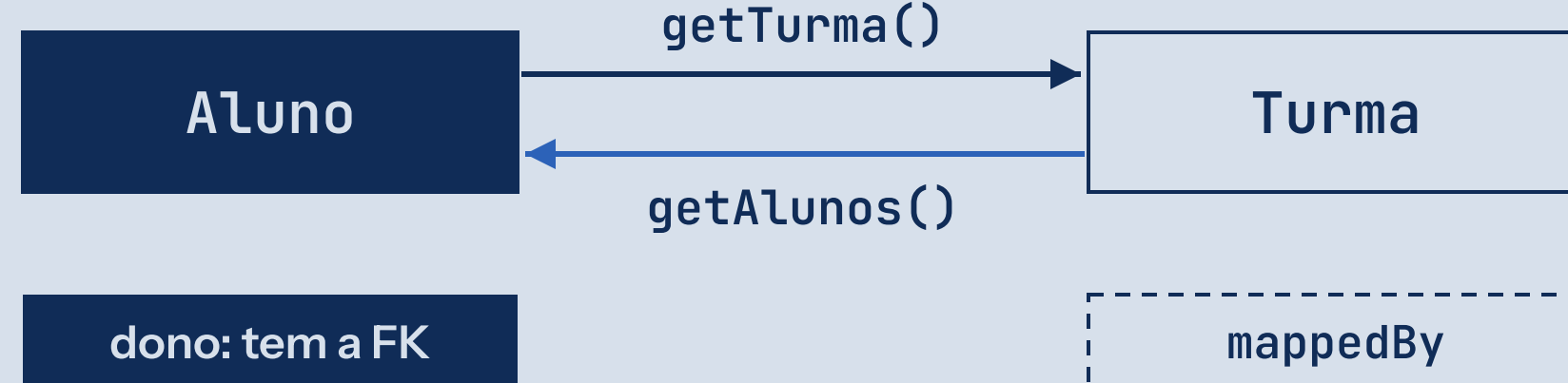
É o diário de classe: só registra quem cursa o quê.

Direção e *dono* do relacionamento

UNIDIRECIONAL



BIDIRECIONAL



Uni: só um lado conhece o outro.

Bi: os dois se conhecem.

CUIDADO

No bidirecional, atualize **os dois lados** em Java.

O **dono** é o lado com a FK na tabela. O outro lado usa `mappedBy` para não criar coluna repetida.

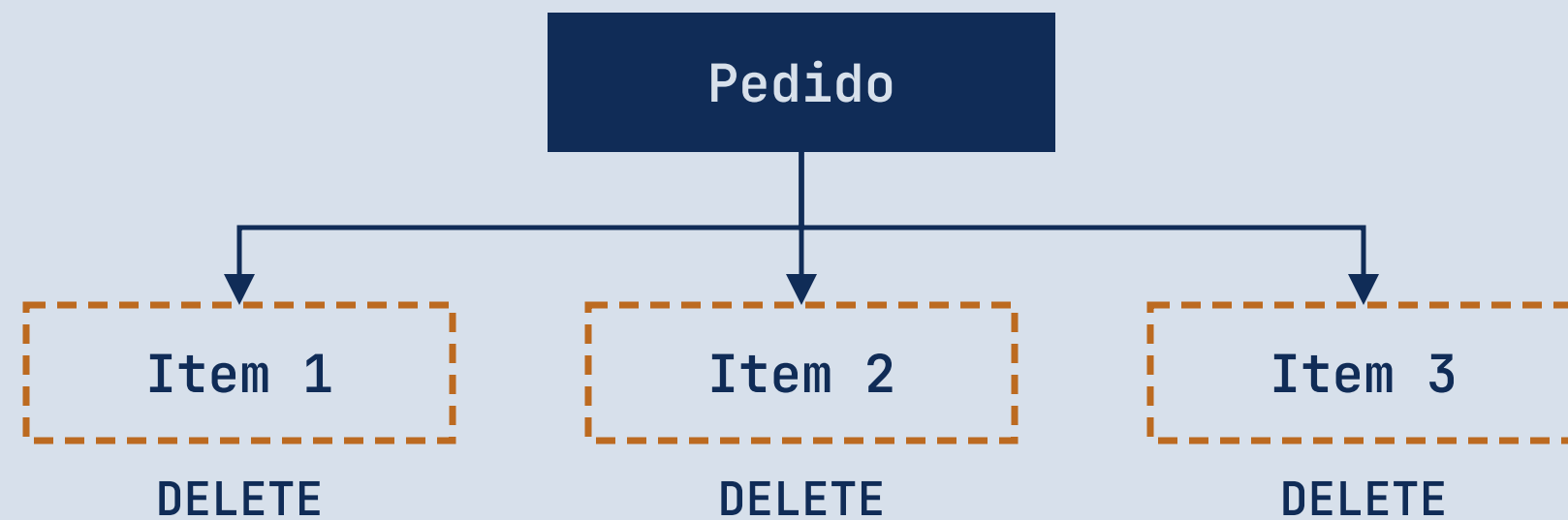
ALEGORIA

Só um cartório registra a escritura; os outros consultam.

Cascata e órfãos

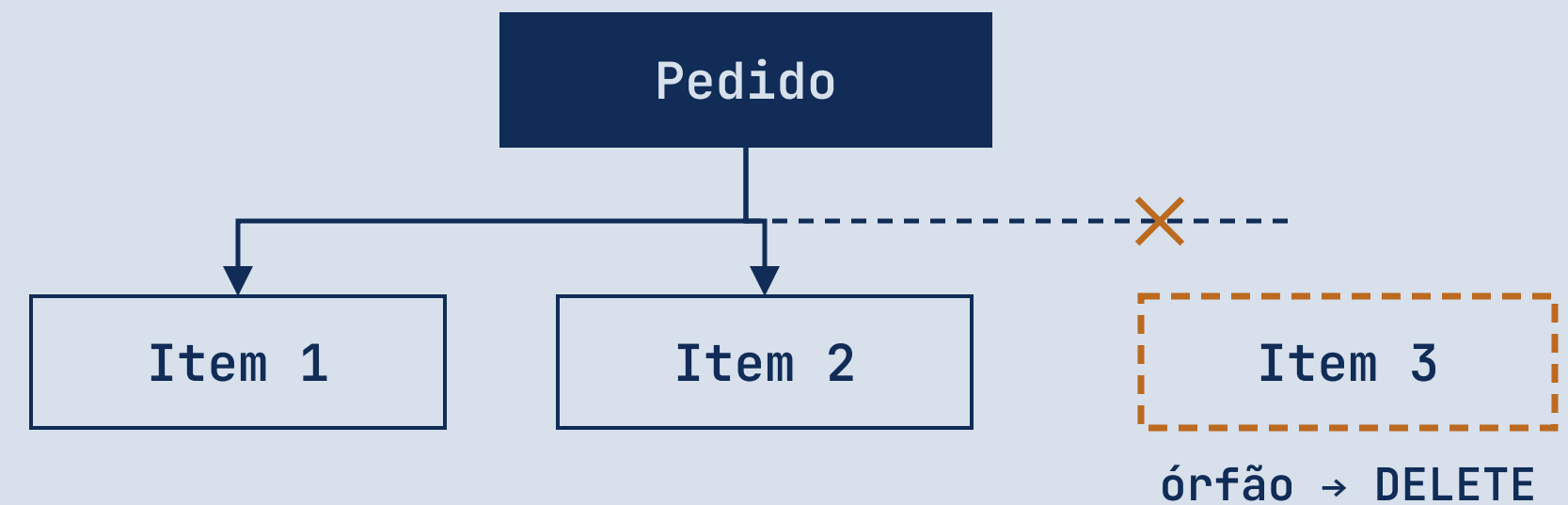
cascade = ALL

`em.remove(pedido)`



orphanRemoval = true

`pedido.getItems().remove(item3)`



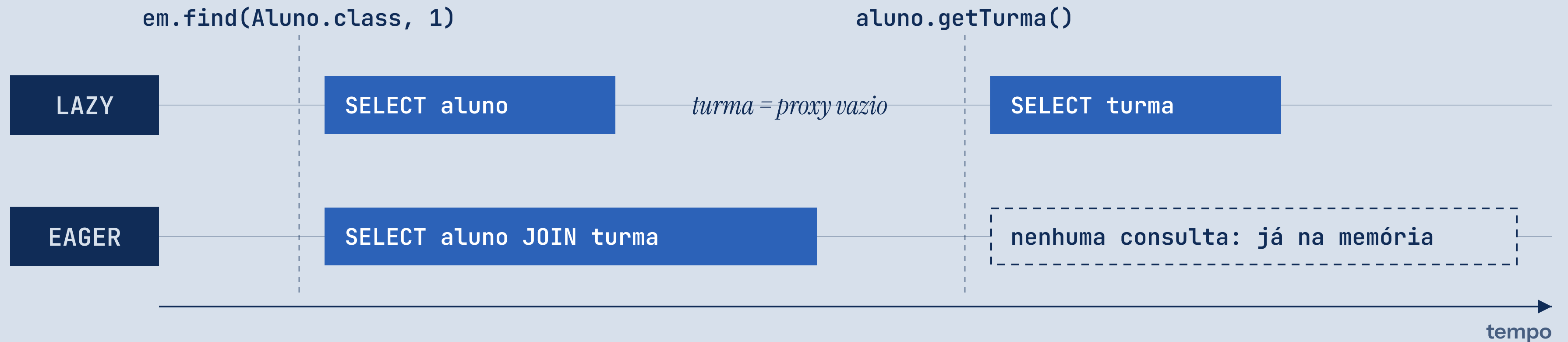
PEDIDO.JAVA

TUDO JUNTO

```
@OneToMany(mappedBy = "pedido",
            cascade = CascadeType.ALL, orphanRemoval = true)
private List<ItemPedido> itens = new ArrayList<>();
```

Alegoria: apagar a pasta apaga os arquivos que estão dentro.

Carregar agora ou *só quando precisar*?



LAZY: *o episódio só carrega quando você dá play*

- + economiza consultas e memória
- erro se a sessão já fechou

EAGER: *baixa a temporada inteira antes da viagem*

- + dados sempre à mão
- traz o que talvez nunca seja usado

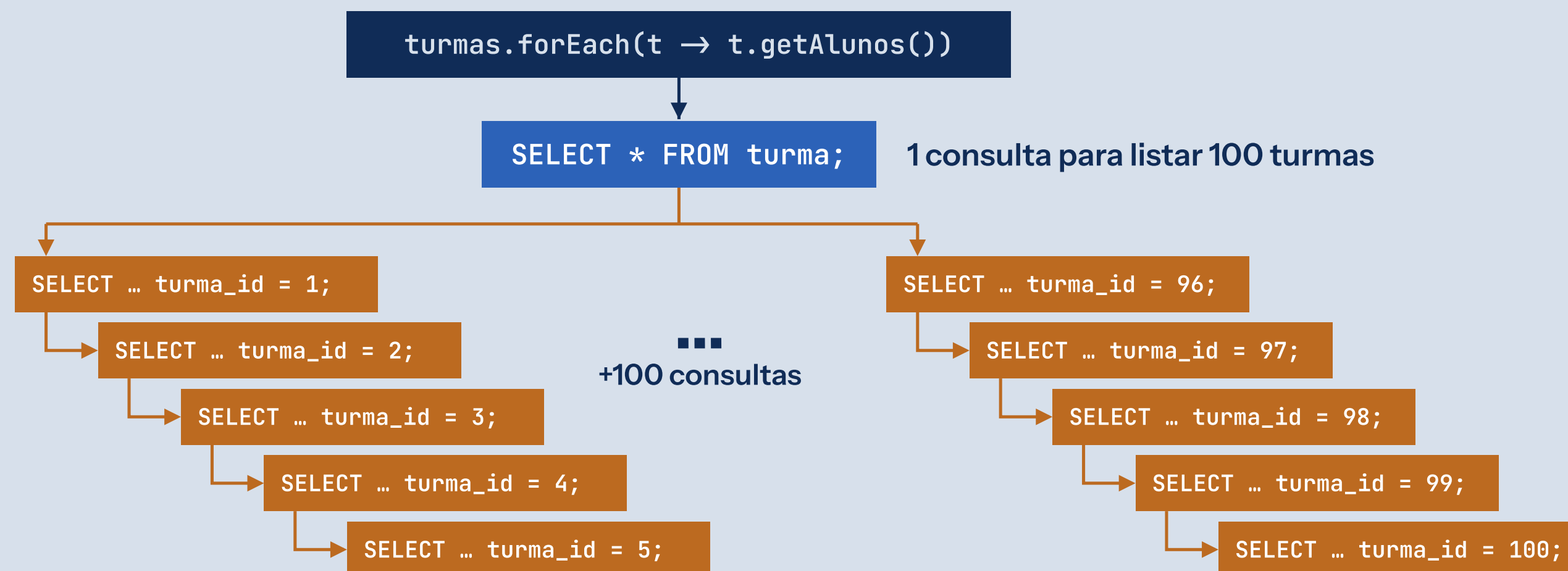
PADRÃO JPA

...ToOne → EAGER

...ToMany → LAZY

Boa prática: tudo LAZY + JOIN FETCH

A ilusão da abstração: o problema $N+1$



A causa

Um laço dispara, sem ninguém ver, uma consulta por item, em vez de uma única consulta com JOIN.

O efeito

Cada ida ao banco soma tempo de rede: com muitos itens, a tela fica lenta. É o gargalo mais comum de um ORM mal configurado.

O que acontece *numa transação*

MATRICULAR.JAVA

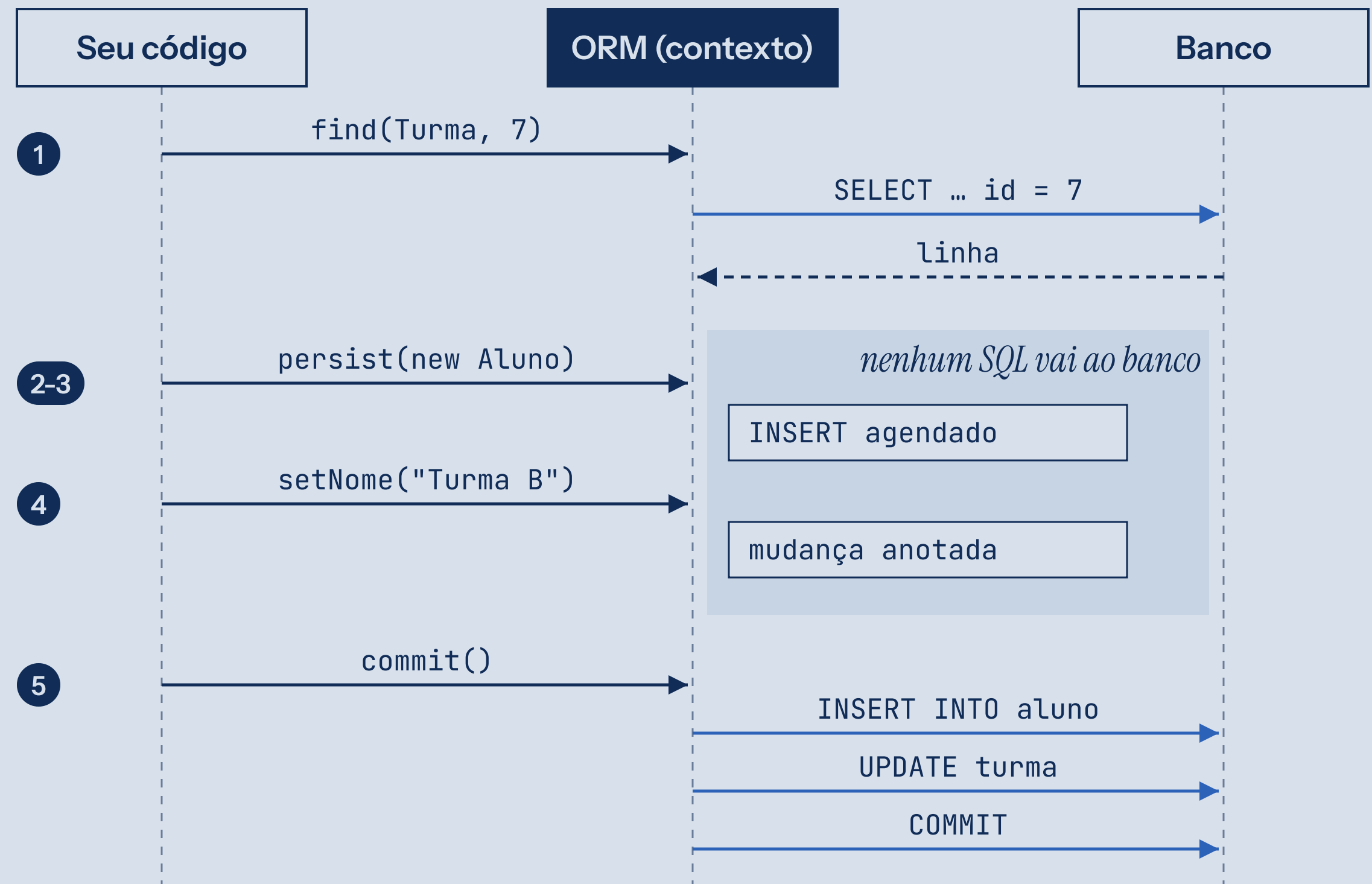
TRANSAÇÃO

```
em.getTransaction().begin();  
1 Turma t =  
    em.find(Turma.class, 7L);  
2 Aluno a = new Aluno("Ana");  
    a.setTurma(t);  
3 em.persist(a);  
4 t.setNome("Turma B");  
5 em.getTransaction().commit();
```

Atenção: vale para chaves por SEQUENCE. Com IDENTITY, o banco precisa gerar o id, então o INSERT sai já no persist().

UNIT OF WORK

Um carrinho de compras: o ORM anota tudo e só passa no caixa no commit.

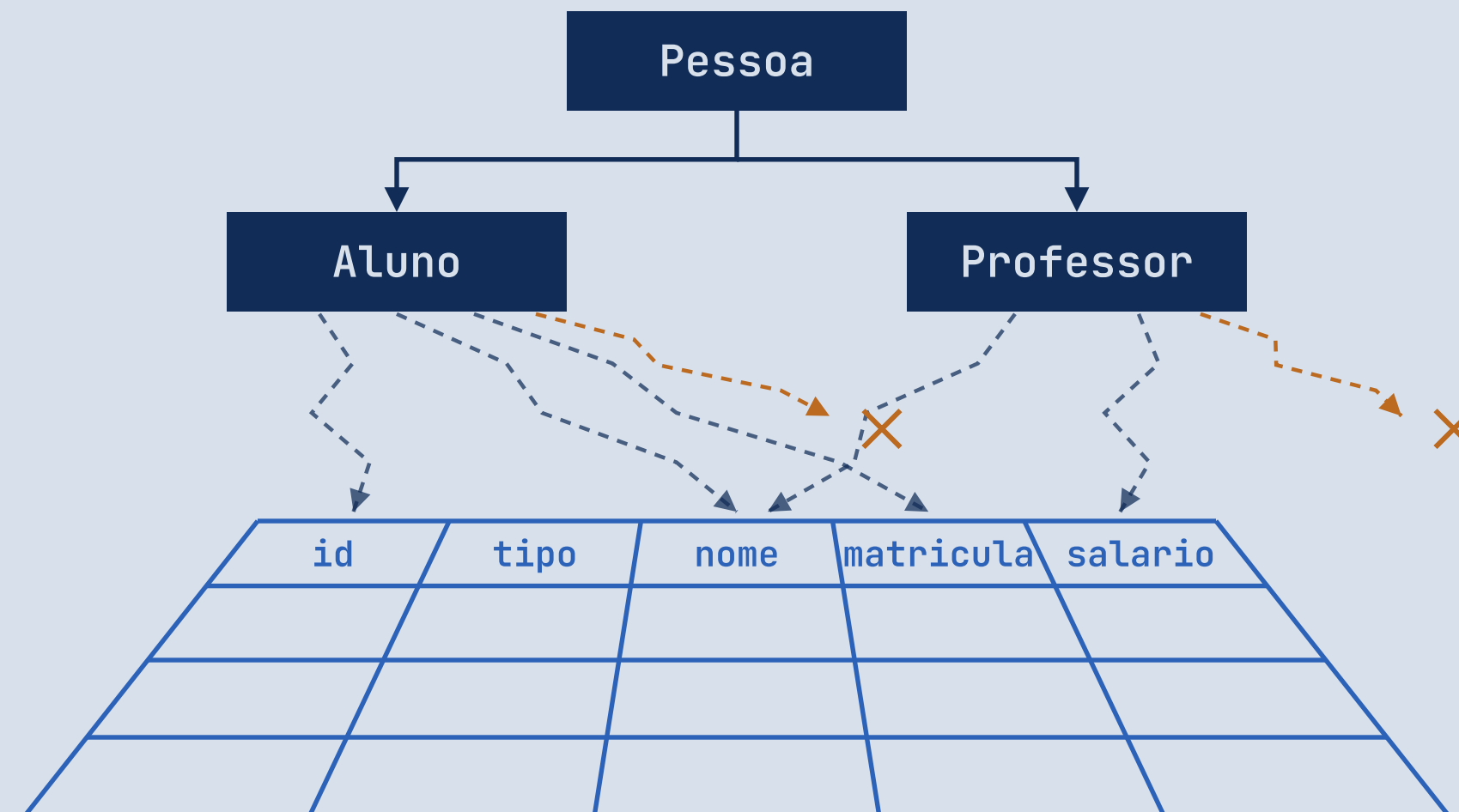


PARTE 06 Além do básico

O desafio da *herança* no banco relacional

A abstração (POO)

Herança é nativa: Aluno e Professor reaproveitam tudo de Pessoa, de forma elegante.



A limitação (banco)

O modelo relacional não tem herança: só tabelas, colunas e chaves.

A MISSÃO DO ORM

Achatar a árvore de herança em tabelas **sem perder integridade**, **sem redundância exagerada** e **sem destruir o desempenho** de leitura.

Três estratégias para mapear a *herança*

Tabela única

SINGLE_TABLE

pessoa	
id	PK
tipo	
nome	
matricula	
salario	

discriminador

NULL para quem não usa a coluna

Uma tabela para a família toda.

+ leitura rápida, zero JOIN

- colunas NULL para quem não usa

Por subclasse

JOINED

pessoa	
id	PK
nome	

aluno	
id	PK-FK
matricula	

professor	
id	PK-FK
salario	

Tabelas normalizadas ligadas por FK.

+ sem NULL, boa integridade

- JOIN em toda consulta

Por classe

TABLE_PER_CLASS

aluno	
id	PK
nome	
matricula	

professor	
id	PK
nome	
salario	

nome repetido nas duas

Uma tabela por classe concreta.

+ tabelas independentes

- colunas repetidas; “todas as pessoas” pede UNION

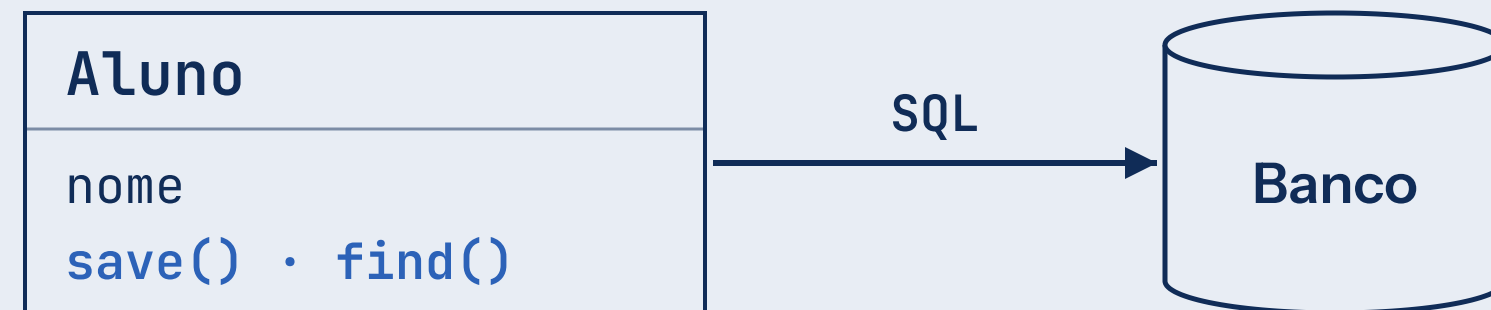
ORM · PERSISTÊNCIA DE DADOS

SEMINÁRIO 01

Active Record *vs* Data Mapper

O objeto *se salva*

RAILS · DJANGO



```
aluno = Aluno.new(nome: "Ana")
aluno.save    # ele mesmo grava
```

O aluno leva a própria ficha à secretaria e a arquiva.

Alguém *salva por ele*

JPA · EF CORE

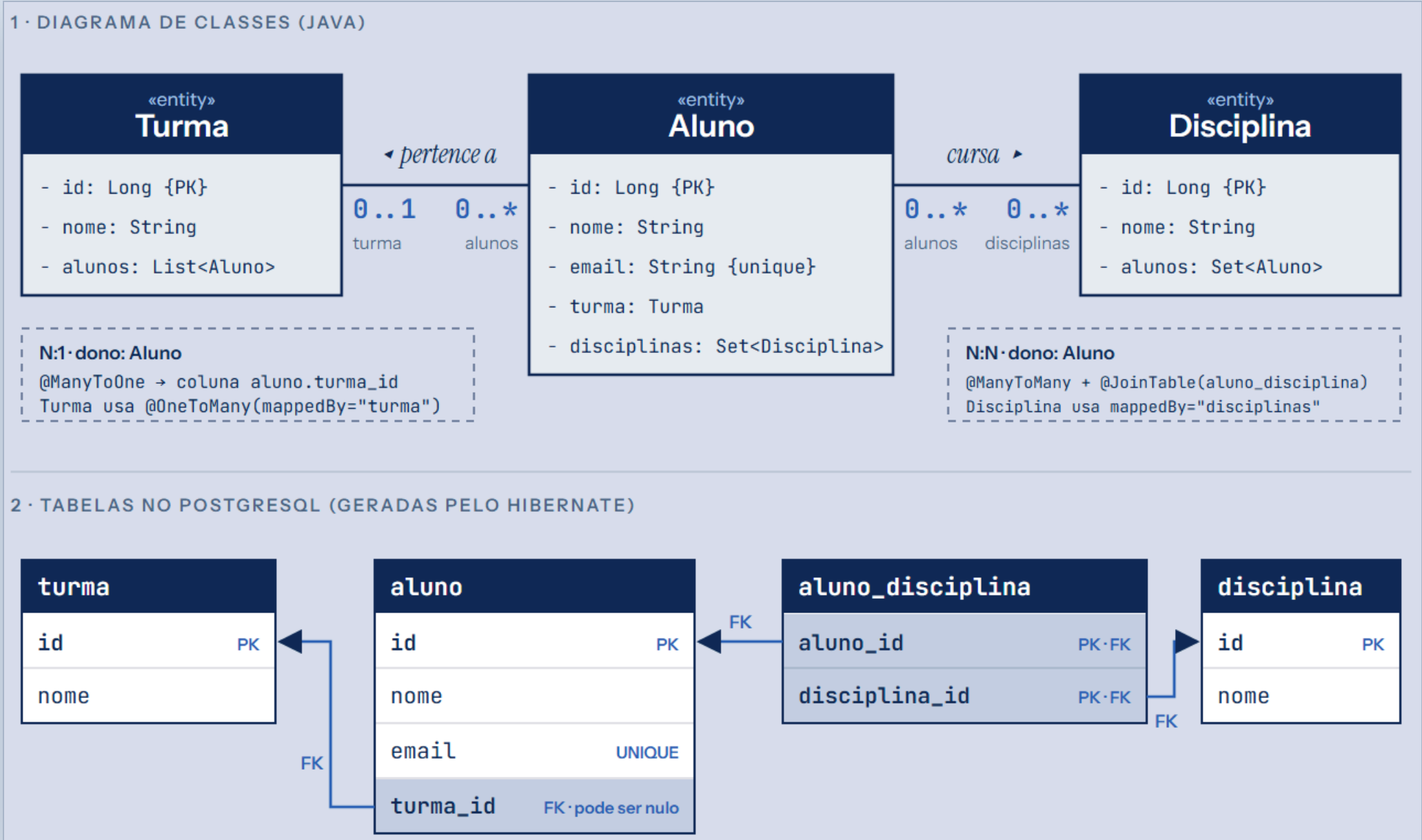


```
Aluno aluno = new Aluno("Ana");
em.persist(aluno); // outro grava
```

O aluno só preenche; a secretaria arquiva.

PARTE 07 Demonstração · Matrícula escolar

Nosso projeto: três entidades, *dois relacionamentos*



Turma $1:N$ Aluno

Cada aluno tem **no máximo uma** turma: a coluna `aluno.turma_id`.

Aluno $N:N$ Disciplina

Um aluno cursa **várias** disciplinas: uma linha em `aluno_disciplina` para cada.

O BANCO RECUSA

o mesmo aluno em duas turmas · a mesma disciplina duas vezes · dois alunos com o mesmo e-mail

Java 17 · Hibernate 7.4 · PostgreSQL 17 · Swing

Configuração e *camadas* do projeto

META-INF/PERSISTENCE.XMLJPA

```
<persistence-unit name="escola">
  <class>modelo.Aluno</class>
  <class>modelo.Turma</class>
  <class>modelo.Disciplina</class>
  <properties>
    <property value="drop-and-create"
      name="jakarta.persistence.schema-generation
        .database.action"/>
    <property name="hibernate.show_sql" value="true"/>
  </properties>
</persistence-unit>
```

- **drop-and-create:** a cada abertura, o Hibernate recria as tabelas a partir das classes.
- URL, usuário e senha vêm de db.properties.

PACOTE	RESPONSABILIDADE
modelo	Aluno, Turma, Disciplina (entidades)
persistencia	fábrica do JPA, conexão, captura do SQL
negocio	os passos da demonstração (Escola)
visao	a janela em Swing

PERSISTENCIA/MONITORSQL.JVAO PLACAR

```
public class MonitorSQL implements StatementInspector {
  public String inspect(String sql) {
    contador++;           // placar da tela
    avisarPainel(sql);    // painel da direita
    return sql;           // segue para o banco
  }
}
```

O problema N+1 *na tela*

4 · Laço com LAZY

N+1

5 · JOIN FETCH

1 consulta

Código Java executado

```
List<Turma> turmas = em.createQuery(
    "SELECT t FROM Turma t", Turma.class)
    .getResultList();           // 1 consulta
for (Turma t : turmas) {
    t.getAlunos().size();       // +1 por turma!
}
```

SQL enviado ao banco

11

gerado pelo Hibernate

comandos neste passo

Código Java executado

```
List<Turma> turmas = em.createQuery(
    "SELECT DISTINCT t FROM Turma t " +
    "LEFT JOIN FETCH t.alunos", Turma.class)
    .getResultList();           // 1 consulta só
for (Turma t : turmas) {
    t.getAlunos().size();       // já está na memória
}
```

SQL enviado ao banco

1

gerado pelo Hibernate

comandos neste passo

SQL ENVIADO

11 COMANDOS

```
SELECT ... FROM turma ORDER BY id           -- 1
SELECT ... FROM aluno WHERE turma_id = ?    -- +1
... mais 9 vezes, uma por turma
```

SQL ENVIADO

1 COMANDO

```
SELECT DISTINCT t.id, t.nome, a.id, a.nome, ...
FROM turma t
LEFT JOIN aluno a ON t.id = a.turma_id
```

Mesmo resultado na tela, **11 idas ao banco contra 1**. É o problema explicado no slide 29, agora no nosso projeto.

O que o ORM traz *de bom*

Produtividade Nada de copiar coluna por coluna.	Pensar em <i>objetos</i> Aluno e Turma, não linhas e colunas.	Portabilidade Trocar de banco muda o dialeto, não o código.
Segurança Parâmetros vinculados contra SQL injection.	Cache e transações Não relê o que já tem; agrupa gravações.	Um só <i>lugar</i> O mapeamento fica junto da classe.

Onde o ORM pode *atrapalhar*

01 Abstração que vaza Sem ler o SQL gerado, não dá para saber por que ficou lento.	02 Consultas N+1 Laços que disparam uma consulta por item.	03 Curva de aprendizado Sessão, ciclo de vida, cascata, carregamento.
04 Relatórios pesados Milhões de linhas: SQL direto é melhor.	05 Memória e CPU Transformar cada linha em objeto custa.	06 “Mágica” demais Cascata e dirty checking geram SQL escondido.

DICA

Ligue o log de SQL (`spring.jpa.show-sql=true`) e leia o que o ORM envia.

A escolha da ferramenta: ORM *vs* SQL nativo

← Controle e desempenho máximo (SQL)

Produtividade e integração (ORM) →



o padrão para a maioria dos sistemas

SQL nativo



VANTAGENS

Controle total das consultas; máximo desempenho em operações pesadas.

CENÁRIOS IDEAIS

Relatórios e cargas em massa; consultas legadas complexas.

O PREÇO

Código repetitivo e refatoração frágil.

ORM



VANTAGENS

Muito mais produtividade; integra naturalmente com orientação a objetos.

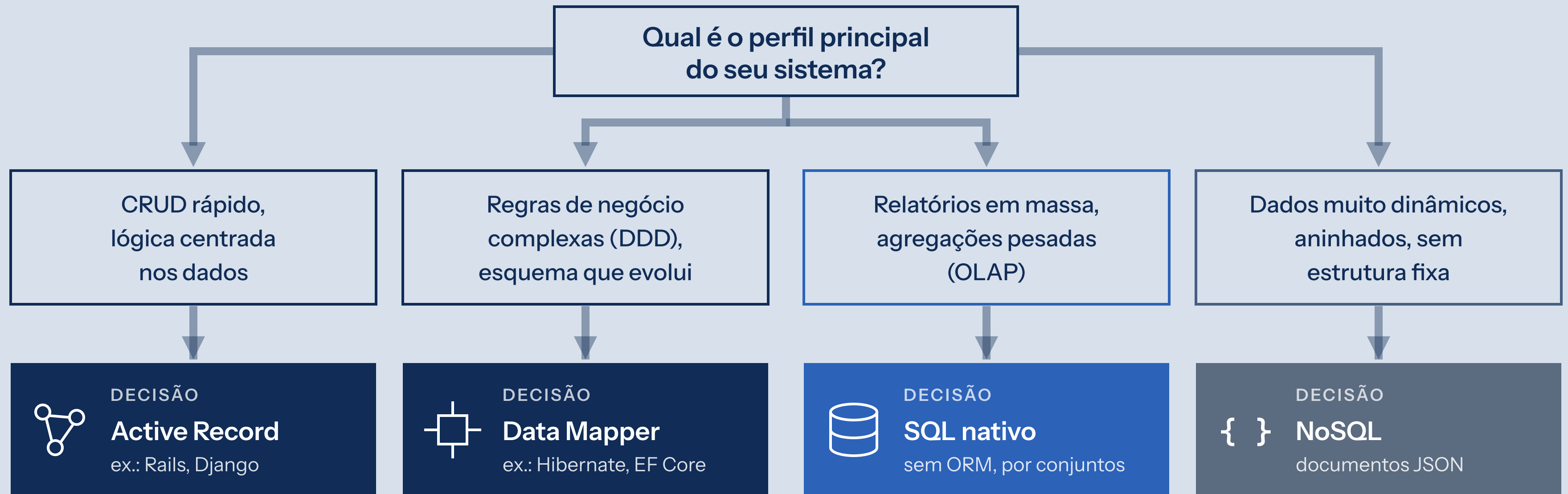
CENÁRIOS IDEAIS

CRUD e cadastros; domínios com muitas regras e classes.

O PREÇO

Risco de N+1 e SQL ineficiente se mal configurado.

Síntese: a *matriz de decisão*



O ORM é indispensável, mas **nunca substitui** conhecer bem o banco relacional e otimizar consultas.

Os conceitos *em uma tela*

CONCEITO	O QUE É	EM JPA
ORM	Traduz objetos ↔ tabelas	Hibernate, EclipseLink
Entidade	Classe com identidade → tabela	@Entity
Identificador	Atributo → chave primária	@Id, @GeneratedValue
Atributo	Campo ↔ coluna	@Column, @Transient
Objeto embutido	Classe de valor sem tabela	@Embeddable, @Embedded
Relacionamento	Referência ↔ chave estrangeira	@OneToMany, @ManyToOne...
Dono	Lado que guarda a FK	@JoinColumn · mappedBy
Carregamento	Quando os associados são lidos	FetchType.LAZY / EAGER
Impedância	As diferenças entre os modelos	<i>o motivo de tudo isso</i>

Obrigado!

Perguntas?

O ORM não substitui saber SQL: ele deixa você usar SQL *só quando realmente importa*.

REFERÊNCIAS

FOWLER, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

BAUER, C.; KING, G.; GREGORY, G. *Java Persistence with Hibernate*. 2. ed. Manning, 2015.

ECLIPSE FOUNDATION. *Jakarta Persistence 3.1 Specification*, 2022. Documentação do Hibernate ORM.